Lab 2: Recipe Finder App

Objective: Build a Kotlin Android app using Jetpack Compose that consumes a **simplified JSON SERVER API** to create a recipe finder application. Students will practice modern Android development (Kotlin, MVVM, Jetpack Compose, Hilt, etc.) with clear minimum requirements.

Score: 15 points

1. What You'll Build

An Android recipe app where users can:

- Browse popular meals, categories, and cuisines (areas)
- Get random meal suggestions
- View detailed recipe information (ingredients, instructions, image, etc.)
- Save favorite recipes locally
- Explore meals filtered by categories
- Navigate across app sections using bottom navigation

Think of it like a **lightweight digital cookbook**.

2. API Documentation

We will use a **pre-configured JSON API** (powered by json-server).

Endpoints:

```
curl --location 'https://meal-db-sandy.vercel.app/meals' \
--header 'X-DB-NAME: d443dd4e-ac1d-4d5e-9885-cf05682f0ab9'

curl --location 'https://meal-db-sandy.vercel.app/categories' \
--header 'X-DB-NAME: d443dd4e-ac1d-4d5e-9885-cf05682f0ab9'
```

NOTE: `X-DB-NAME` header only accepts a GUID format. Each student must use their own unique GUID to ensure they have a dedicated database. This prevents interference from other students when using the API. Use this website for GUID generation: https://www.guidgenerator.com/

Tip for students: Since this is powered by json-server, you can explore flexible queries (filtering, sorting, pagination, etc.) for your own creative extensions. See: json-server docs.

3. Minimum Feature Requirements

Your app must include at least the following features:

1. **Onboarding Screens**
    - Simple introduction screens before entering the app.
2. **Home Screen**
    - Show popular meals.
    - Show random food suggestion.
    - Display **category cards** and **area cards**.
    - Navigation from sections → corresponding meal list.
    - Bottom navigation with **Home, Explore, Favourite**.
3. **Explore Screen**
    - Show categories as filter chips.
    - Filtered meal list displayed below.
4. **Meal Detail Screen**
    - Display **all available fields** from the meal object (clean UI).
    - Favorite/unfavorite option.
5. **Favourite Screen**
    - Save favorites locally using **Room DB**.
    - List all saved meals.

4. Technical Requirements

You must use these specific technologies in your implementation:

Core Technologies:

- **Kotlin** - Primary programming language
- **Jetpack Compose** - Modern UI toolkit for building native Android UI
- **MVVM Architecture Pattern** - Separate UI, business logic, and data layers
- **Coroutines** - Handle asynchronous operations and threading
- **Hilt Dependency Injection** - Manage dependencies throughout the app

Additional Required Libraries:

- **Retrofit** - For REST API communication
- **StateFlow** - Reactive state management in ViewModels
- **Navigation Compose** - Handle navigation between screens
- **Coil or Glide** - Image loading and caching
- **DataStore or SharedPreferences** - Local data storage for favorites

5. Evaluation Criteria (15 points)

- **Minimum Feature Requirement** → 8 points
- **Required Technical Implementation** → 5 points
- **Submission Requirement** → 2 points
- **Extra Mile & Creativity** → +2 bonus points

6. Submission Requirements

- **App demo video**: Walk through all implemented features. Your video should be **maximum 2 minutes long**
- **Private Github repo**:
  - Name format: recipe-app-android-[class]-[name]
  - Add me hangsopheak as collaborator as collaborator.
- **Add Github repository link to the submission**

Note: Copying from others is not allowed. If you are caught copying, you will receive a zero score. Please complete the work on your own.